

AI Matching Pipeline

Performance, Scalability & Optimisation Report

Prepared for: Dyversifying Ltd | Date: 17 September 2026 | Classification: Confidential

1. Executive Summary

This report documents the measured performance of the Dyversifying AI Matching Pipeline, analyses how processing time scales with increasing data volumes, and presents a prioritised optimisation roadmap. The findings are intended to inform product and infrastructure decisions ahead of the platform's public launch.

Key Finding

The current pipeline is fully functional for offline batch processing and incremental production matching. A full re-run over all data takes approximately 7 seconds per candidate-job pair. In live operation, only NEW pairs are ever processed, reducing typical response times to 2-12 minutes per event. Parallel API execution (planned) will reduce this by 5-10x.

2. Measured Performance (17 September 2026)

Test Configuration

Candidates (CVs):	22 (including 4 scanned/OCR-processed PDFs)
Job Descriptions:	8 active roles
Total pairs evaluated:	176 (22 x 8)
Pipeline stages:	3 (Profession Gate -> Requirements Score -> Deep Analysis)

Timing Breakdown

Total elapsed time:	20 minutes 36 seconds
Average per pair:	~7.0 seconds
Gemini API calls made:	~205 (176 Stage 1 + 15 Stage 2 + 14 Stage 3)
Average per API call:	~6.0 seconds (includes network latency + model inference)

Funnel Results

Stage	Input	Output	Pass Rate	Avg. Time
Stage 1 - Profession Gate	176 pairs	15 passed	8.5%	~3.5s
Stage 2 - Requirements Score	15 pairs	15 scored	100%	~4.0s
Stage 3 - Deep Analysis	14 top cand.	9 Possible	-	~8.0s

3. Scalability Analysis

The table below shows projected durations for a FULL batch re-run across all pairs. In production this scenario NEVER occurs - matching is always incremental (see Section 4). These figures represent worst-case numbers, shown here for full transparency.

CVs	Active JDs	Total Pairs	Est. Duration (Full Batch)	Risk
22	8	176	~21 min (measured)	LOW
50	8	400	~47 min	LOW
100	20	2,000	~3.9 h	MEDIUM
200	20	4,000	~7.8 h	HIGH
500	50	25,000	~49 h	CRITICAL
1,000	50	50,000	~98 h (~4 days)	CRITICAL

Important Context

These batch figures assume the current sequential API architecture. With parallel execution (Optimisation #1 in the roadmap), all durations would be reduced by a factor of 5 to 10. Additionally, the incremental production model means the platform will never perform a full re-run in normal operation.

4. How Matching Works in Production

A critical distinction: the 20-minute batch run today was a one-time setup to process all historical data. In live operation, the system runs in INCREMENTAL mode - only new, unprocessed pairs are ever evaluated.

Trigger: Recruiter posts a new Job Description

-> System matches 1 JD against all existing CVs
e.g. 100 CVs x 1 JD = 100 pairs -> approx. 12 minutes

Trigger: New candidate uploads CV

-> System matches 1 CV against all active JDs
Example: 1 CV x 20 JDs = 20 pairs -> approx. 2-3 minutes

Implicit Feedback Loop (already implemented)

Recruiter actions on the platform (shortlist, invite to interview, dismiss) are automatically captured via the POST /api/v1/matching/feedback endpoint. These signals continuously improve the ML calibration model in the background - without recruiters ever knowing they are training the system. This is by design.

5. Optimisation Roadmap

The following improvements are planned in priority order. Items 1 and 2 alone would reduce processing time by 90% for large-scale deployments.

#1 Parallel / Async API Calls

Priority: HIGH Effort: Medium 5 - 10x

Stage 1 (Profession Gate) currently runs sequentially. Switching to async concurrent execution would allow all pairs for a given JD to be evaluated simultaneously, bound only by API rate limits.

#2 Stage 1 Result Caching

Priority: HIGH Effort: Low 2x

A candidate's profession domain rarely changes. Stage 1 results can be cached per (CV, JD-domain) pair and reused across future matching runs without an API call.

#3 Stage 3 On-Demand Only

Priority: MEDIUM Effort: Low 1.5x

Deep analysis (Stage 3) could be deferred until a recruiter actually opens a candidate profile, rather than pre-computed for all top candidates. This reduces upfront cost.

#4 Gemini Batch API

Priority: MEDIUM Effort: Low 3 - 5x

Google's Batch API allows multiple prompts to be submitted as a single asynchronous job, significantly reducing per-call overhead for large volumes.

#5 Real Recruiter Labels -> Retrain ML Model

Priority: HIGH Effort: Ongoing Quality

As recruiters interact with results (shortlist/dismiss), the ML calibration model should be retrained periodically. This improves match quality over time without changing infrastructure.

6. Conclusions & Recommendations

1. The system is production-ready today

Incremental matching means typical recruiter-facing wait times are 2-12 minutes, which is acceptable for an asynchronous background process.

2. Parallel execution is the #1 priority

A relatively low-effort code change to make Stage 1 calls concurrent would immediately reduce matching time by 5-10x, making the system viable at 500+ CVs.

3. Real recruiter data is essential long-term

The current ML model was bootstrapped with AI-generated labels. As the platform grows, replacing these with genuine recruiter decisions is critical for model accuracy.

4. Batch re-runs should be avoided in production

Full re-runs (as performed today during setup) should only occur when the matching algorithm itself is updated. All other matching is incremental.